ADVANCED PYTHON PROGRAMMING

PORTFOLIO

Name: VIKNESH G

Reg. No: 23MID0161

Q.No	Question
1	Interpreter vs compiler
2	append() vs extend()
3	Attributes and methods of a frozenset
4	List unpacking
5	Shallow copy vs deep copy
6	sorted() function
7	List comprehension
8	concurrent.futures module
9	zip() function

Question 1

What is the difference between an interpreter and a compiler?

Answer

Both are translators. They convert the program we write (high-level code) into machine code that the computer understands.

- **Compiler:** translates the full program first, then runs it. It creates an executable file. Examples: C, C++.
- **Interpreter:** translates and runs the program line by line. No separate file is created. Examples: Python, PHP.

Point	Compiler	Interpreter
Translation	Whole program at once	One line at a time
Errors	Shown after checking the whole program	Shown when that line is reached
Speed	Faster to run	Slower to run
Output file	Creates an executable file	No executable file

In the program below, Python runs the first lines and prints their output. It stops only when it reaches the line with the error. This is how an interpreter works. (The error message at the end is expected.)

Program

```
# Python is an interpreted language: lines run one by one
price = 250
quantity = 4
print("Price    :", price)
print("Quantity :", quantity)
print("Total    :", price * quantity)

# This line has an error, so the program stops here.
# The three lines above have already run.
print("Discount :", price * discount)
```

Screenshot of the code

```
[1]: # Python is an interpreted language: lines run one by one
     price = 250
     quantity = 4
     print("Price    :", price)
     print("Quantity :", quantity)
     print("Total    :", price * quantity)

     # This line has an error, so the program stops here.
     # The three lines above have already run.
     print("Discount :", price * discount)
```

Output

```
Price    : 250
Quantity : 4
Total    : 1000
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 10
      6 print("Total    :", price * quantity)
      7
      8 # This line has an error, so the program stops here.
      9 # The three lines above have already run.
---> 10 print("Discount :", price * discount)

NameError: name 'discount' is not defined
```

Question 2

What is the difference between append() and extend()?

Answer

Both methods add items to the end of a list, but they do it differently.

- **append(x):** adds x as a single item. If x is a list, it becomes a list inside the list (nested list).
- **extend(iterable):** adds every item of the iterable one by one into the list.
- Both change the original list and return None.

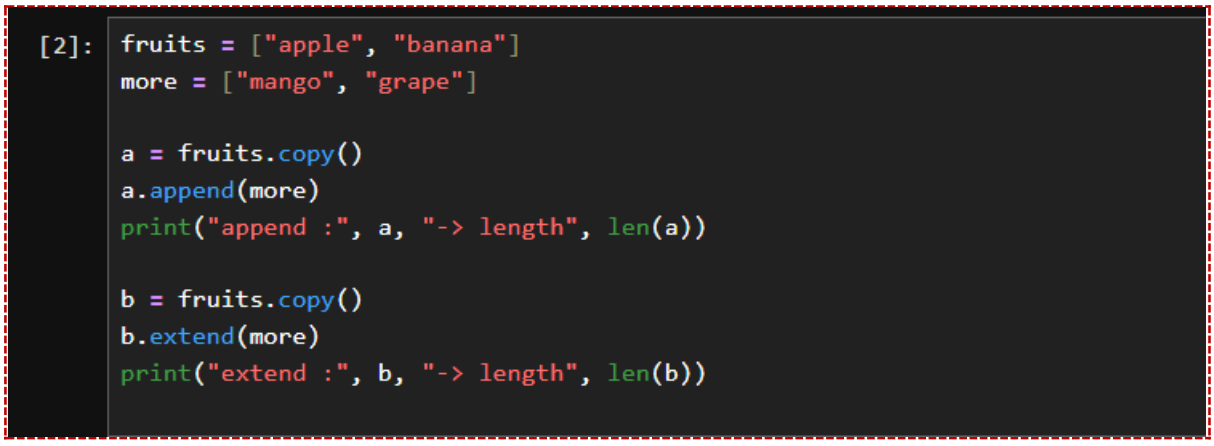
Program

```
fruits = ["apple", "banana"]
more = ["mango", "grape"]

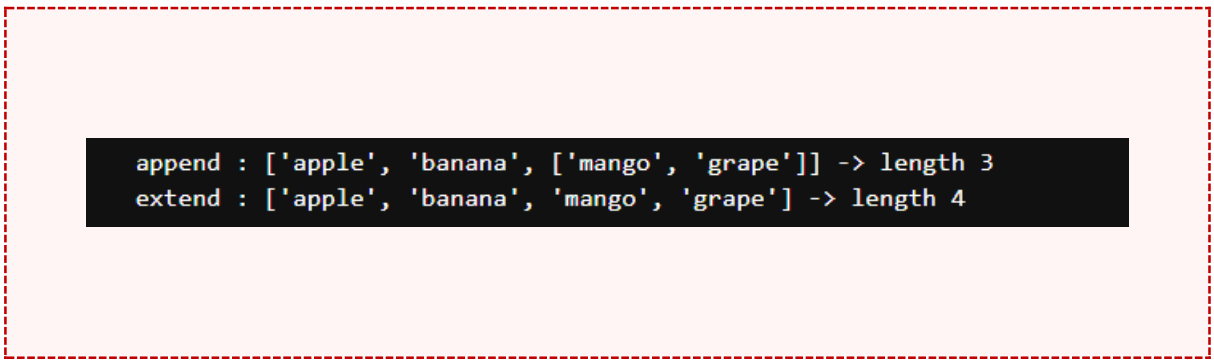
a = fruits.copy()
a.append(more)
print("append :", a, "-> length", len(a))

b = fruits.copy()
b.extend(more)
print("extend :", b, "-> length", len(b))
```

Screenshot of the code



Output



Question 3

What are the attributes and methods of a frozenset?

Answer

A frozenset is like a set, but it cannot be changed after it is created.

- It stores only unique items and has no order (no indexing).
- It is immutable, so add(), remove(), discard(), pop(), clear() and update() are not available.
- It is hashable, so it can be used as a dictionary key or inside another set. A normal set cannot.

Method	Use
union()	All items from both sets
intersection()	Items common to both
difference()	Items in the first but not in the second
symmetric_difference()	Items in either set, but not both
issubset()	True if all items are in the other set
issuperset()	True if it contains all items of the other set
isdisjoint()	True if there is nothing in common
copy()	Returns a copy

Program

```
team_a = frozenset(["Ravi", "Kumar", "Anu", "Mohan"])
team_b = frozenset(["Anu", "Mohan", "Divya"])

print("Union        :", sorted(team_a.union(team_b)))
print("Intersection :", sorted(team_a.intersection(team_b)))
print("Difference   :", sorted(team_a.difference(team_b)))
print("Symmetric    :", sorted(team_a.symmetric_difference(team_b)))
print("Subset?      :", frozenset(["Anu"]).issubset(team_a))
print("Superset?    :", team_a.issuperset(["Anu", "Ravi"]))
print("Disjoint?    :", team_a.isdisjoint(team_b))

# frozenset cannot be changed
try:
    team_a.add("Sita")
except AttributeError as e:
    print("Error :", e)

# frozenset can be a dictionary key
info = {team_b: "Team B"}
print(info[team_b])
```

Screenshot of the code

```
[3]: team_a = frozenset(["Ravi", "Kumar", "Anu", "Mohan"])
     team_b = frozenset(["Anu", "Mohan", "Divya"])

     print("Union        :", sorted(team_a.union(team_b)))
     print("Intersection :", sorted(team_a.intersection(team_b)))
     print("Difference   :", sorted(team_a.difference(team_b)))
     print("Symmetric    :", sorted(team_a.symmetric_difference(team_b)))
     print("Subset?      :", frozenset(["Anu"]).issubset(team_a))
     print("Superset?    :", team_a.issuperset(["Anu", "Ravi"]))
     print("Disjoint?    :", team_a.isdisjoint(team_b))

     # frozenset cannot be changed
     try:
         team_a.add("Sita")
     except AttributeError as e:
         print("Error :", e)

     # frozenset can be a dictionary key
     info = {team_b: "Team B"}
     print(info[team_b])
```

Output

```
Union        : ['Anu', 'Divya', 'Kumar', 'Mohan', 'Ravi']
Intersection : ['Anu', 'Mohan']
Difference   : ['Kumar', 'Ravi']
Symmetric    : ['Divya', 'Kumar', 'Ravi']
Subset?      : True
Superset?    : True
Disjoint?    : False
Error : 'frozenset' object has no attribute 'add'
Team B
```

Question 4

What is list unpacking? Explain with an example.

Answer

List unpacking means assigning the items of a list to separate variables in one statement, instead of using index numbers one by one.

- Items are matched to variables from left to right.
- The number of variables must match the number of items, otherwise Python gives a ValueError.
- A variable with * collects the remaining items as a list. Only one * is allowed.

Program

```
# Basic unpacking
point = [5, 10, 15]
x, y, z = point
print("x =", x, " y =", y, " z =", z)

# Using * to collect the extra values
scores = [90, 80, 70, 60, 50]
top, *others = scores
print("top    =", top)
print("others =", others)

head, *middle, tail = scores
print("head =", head, " middle =", middle, " tail =", tail)

# Wrong number of variables gives an error
try:
    p, q = [1, 2, 3]
except ValueError as e:
    print("Error :", e)
```

Screenshot of the code

[4]:
```
# Basic unpacking
point = [5, 10, 15]
x, y, z = point
print("x =", x, " y =", y, " z =", z)

# Using * to collect the extra values
scores = [90, 80, 70, 60, 50]
top, *others = scores
print("top    =", top)
print("others =", others)

head, *middle, tail = scores
print("head =", head, " middle =", middle, " tail =", tail)

# Wrong number of variables gives an error
try:
    p, q = [1, 2, 3]
except ValueError as e:
    print("Error :", e)
```

Output

```
x = 5  y = 10  z = 15
top    = 90
others = [80, 70, 60, 50]
head = 90  middle = [80, 70, 60]  tail = 50
Error : too many values to unpack (expected 2, got 3)
```

Question 5

What is the difference between shallow copy and deep copy?

Answer

Both make a copy of a list, but they treat nested (inner) lists differently.

- **Shallow copy** (copy.copy): makes a new outer list, but the inner lists are shared with the original. A change inside an inner list appears in both.
- **Deep copy** (copy.deepcopy): copies everything, including inner lists. The copy is fully independent.
- Writing b = a is not a copy. Both names point to the same list.

Program

```
import copy

marks = [[80, 90], [70, 60]]
same = marks                        # same list, new name
shallow = copy.copy(marks)          # new outer list, same inner lists
deep = copy.deepcopy(marks)         # everything copied

marks[0][0] = 100                   # change inside an inner list
print("marks   :", marks)
print("same    :", same)
print("shallow :", shallow)
print("deep    :", deep)

marks.append([50, 40])              # change the outer list
print("\nAfter append")
print("marks   :", marks)
print("shallow :", shallow)
print("deep    :", deep)
```

Screenshot of the code

```
[5]: import copy

marks = [[80, 90], [70, 60]]
same = marks
shallow = copy.copy(marks)
deep = copy.deepcopy(marks)

marks[0][0] = 100
print("marks   :", marks)
print("same    :", same)
print("shallow :", shallow)
print("deep    :", deep)

marks.append([50, 40])
print("\nAfter append")
print("marks   :", marks)
print("shallow :", shallow)
print("deep    :", deep)
```

Output

```
marks   : [[100, 90], [70, 60]]
same    : [[100, 90], [70, 60]]
shallow : [[100, 90], [70, 60]]
deep    : [[80, 90], [70, 60]]

After append
marks   : [[100, 90], [70, 60], [50, 40]]
shallow : [[100, 90], [70, 60]]
deep    : [[80, 90], [70, 60]]
```

Question 6

What is the sorted() function in Python?

Answer

sorted() takes any iterable and returns a new sorted list. The original is not changed.

- **reverse=True** gives descending order.
- **key=** sorts using a rule, for example key=len or key=str.lower.
- **sorted() vs sort():** sorted() returns a new list. list.sort() changes the list itself and returns None.

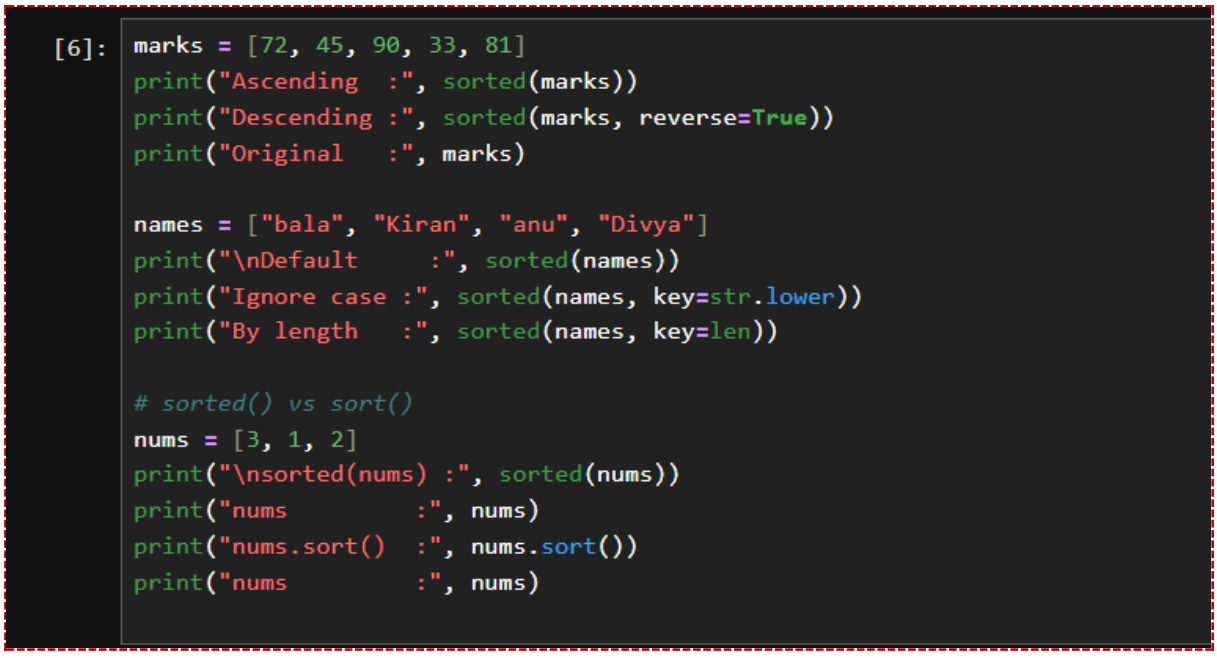
Program

```
marks = [72, 45, 90, 33, 81]
print("Ascending  :", sorted(marks))
print("Descending :", sorted(marks, reverse=True))
print("Original   :", marks)

names = ["bala", "Kiran", "anu", "Divya"]
print("\nDefault     :", sorted(names))
print("Ignore case :", sorted(names, key=str.lower))
print("By length   :", sorted(names, key=len))

# sorted() vs sort()
nums = [3, 1, 2]
print("\nsorted(nums) :", sorted(nums))
print("nums         :", nums)
print("nums.sort()  :", nums.sort())
print("nums         :", nums)
```

Screenshot of the code



Output

```
Ascending  : [33, 45, 72, 81, 90]
Descending : [90, 81, 72, 45, 33]
Original   : [72, 45, 90, 33, 81]

Default     : ['Divya', 'Kiran', 'anu', 'bala']
Ignore case : ['anu', 'bala', 'Divya', 'Kiran']
By length   : ['anu', 'bala', 'Kiran', 'Divya']

sorted(nums) : [1, 2, 3]
nums         : [3, 1, 2]
nums.sort()  : None
nums         : [1, 2, 3]
```

Question 7

What is list comprehension?

Answer

List comprehension is a short way to create a list in a single line.

- **Syntax:** [expression for item in iterable if condition] (the if part is optional)
- It does the same work as a for loop with append(), but the code is shorter.
- For long or complicated logic, a normal for loop is easier to read.

Program

```
# Using a loop
squares = []
for n in range(1, 6):
    squares.append(n * n)
print("Loop          :", squares)

# Same thing using list comprehension
squares2 = [n * n for n in range(1, 6)]
print("Comprehension :", squares2)

# With a condition
odd = [n for n in range(1, 11) if n % 2 != 0]
print("Odd numbers   :", odd)

# With if-else
result = ["Pass" if m >= 40 else "Fail" for m in [72, 35, 58]]
print("Result        :", result)
```

Screenshot of the code

PASTE YOUR CODE SCREENSHOT HERE (from your editor)

(delete this text after pasting)

Output

PASTE YOUR OUTPUT SCREENSHOT HERE (after running the program)

(delete this text after pasting)

Question 8

What is the use of the concurrent.futures module in Python?

Answer

It lets us run several tasks at the same time using a pool of threads or processes, without managing the threads ourselves.

- **ThreadPoolExecutor:** runs tasks in threads (good for waiting tasks like downloads or file reading).
- **ProcessPoolExecutor:** runs tasks in separate processes (good for heavy calculations).
- **submit():** runs one task and returns a Future (a result that may not be ready yet).
- **result():** waits for the task and gives its result.
- **map():** runs a function on every item and returns results in the same order.

In the program below each task waits 1 second. Because they run together, the total time is about 2 seconds, not 5.

Program

```
from concurrent.futures import ThreadPoolExecutor
import time

def double(n):
    time.sleep(1)
    return n * 2

numbers = [1, 2, 3, 4]

start = time.time()
with ThreadPoolExecutor(max_workers=4) as pool:
    f1 = pool.submit(double, 10)
    print("submit result :", f1.result())

    results = list(pool.map(double, numbers))
    print("map results   :", results)

print("Time taken    :", round(time.time() - start), "seconds")
```

Screenshot of the code

```
[7]: from concurrent.futures import ThreadPoolExecutor
     import time

     def double(n):
         time.sleep(1)
         return n * 2

     numbers = [1, 2, 3, 4]

     start = time.time()
     with ThreadPoolExecutor(max_workers=4) as pool:
         f1 = pool.submit(double, 10)
         print("submit result :", f1.result())

         results = list(pool.map(double, numbers))
         print("map results   :", results)

     print("Time taken    :", round(time.time() - start), "seconds")
```

Output

```
submit result : 20
map results   : [2, 4, 6, 8]
Time taken    : 2 seconds
```

Question 9

What is the use of zip() in Python?

Answer

zip() joins two or more iterables item by item and gives tuples. The first items are paired together, then the second items, and so on.

- It returns an iterator, so use list() to see the result.
- It stops at the shortest iterable.
- It is often used in a for loop with unpacking, and to build a dictionary with dict(zip(keys, values)).
- **zip(*pairs)** separates the pairs back into groups (the * unpacks the list of pairs).

Program

```
names = ["Anu", "Bala", "Divya"]
ages = [20, 21, 19]

print(list(zip(names, ages)))

for n, a in zip(names, ages):
    print(n, "is", a, "years old")

# Make a dictionary
print(dict(zip(names, ages)))

# Different lengths: stops at the shortest
print(list(zip([1, 2, 3], ["a", "b"])))

# Separate the pairs again
pairs = [("x", 1), ("y", 2)]
letters, numbers = zip(*pairs)
print(letters, numbers)
```

Screenshot of the code

```
[8]: names = ["Anu", "Bala", "Divya"]
     ages = [20, 21, 19]

     print(list(zip(names, ages)))

     for n, a in zip(names, ages):
         print(n, "is", a, "years old")

     # Make a dictionary
     print(dict(zip(names, ages)))

     # Different lengths: stops at the shortest
     print(list(zip([1, 2, 3], ["a", "b"])))

     # Separate the pairs again
     pairs = [("x", 1), ("y", 2)]
     letters, numbers = zip(*pairs)
     print(letters, numbers)
```

Output

```
[('Anu', 20), ('Bala', 21), ('Divya', 19)]
Anu is 20 years old
Bala is 21 years old
Divya is 19 years old
{'Anu': 20, 'Bala': 21, 'Divya': 19}
[(1, 'a'), (2, 'b')]
('x', 'y') (1, 2)
```